PRÁCTICA DE EXAMEN PRÁCTICO EVALUATIVO

Sistema de Inteligencia Táctica HoloNet

Herramientas de Mapeo Objeto-Relacional (Hibernate 6 / JPA)
Estrategia Jerárquica de Persistencia: Tabla Única (SINGLE_TABLE)

Resumen Técnico del Proyecto

Construcción de una arquitectura de persistencia polimórfica en Java con Hibernate 6 y PostgreSQL para resolver el desfase objeto-relacional (*Impedance Mismatch*). Se implementa un modelo jerárquico unificado, asociaciones perezosas y consultas de alto rendimiento en HQL avanzado para el control táctico de la flota.

✔ **Persistencia Polimórfica:** Estrategia `InheritanceType.SINGLE_TABLE` con discriminador dinámico.

✔ **Optimización de Consultas:** Mitigación del problema N+1 mediante `INNER JOIN FETCH`.

✔ **Filtrado Polimórfico en ORM:** Uso de la función `TYPE(n)` y proyecciones tabulares escalares.

✔ **Control Transaccional Seguro:** Gestión de sesiones mediante el patrón *try-with-resources*.

ALUMNO / DESARROLLADOR
Antonio Cano Ruiz

CICLO FORMATIVO
2º Desarrollo de Aplicaciones Multiplataforma (DAM)

MÓDULO PROFESIONAL
Acceso a Datos (Código: 2627)

UNIDAD DE TRABAJO
UT4 - Persistencia (Hibernate 6 / JPA)

1. Introducción y Planteamiento del Problema

En el desarrollo de software empresarial contemporáneo, uno de los desafíos más críticos es el **desfase objeto-relacional (*Object-Relational Impedance Mismatch*)**. Este fenómeno surge al intentar conciliar el paradigma orientado a objetos (que maneja herencia, polimorfismo, encapsulamiento e identidad) con el modelo relacional tradicional (basado en tablas planas, relaciones de claves foráneas y álgebra relacional).

El proyecto **HoloNet** aborda este problema para el departamento de inteligencia militar, que requiere clasificar y controlar el censo de activos navales interceptados en el Borde Exterior (cazas ligeros y destructores estelares) asignados a distintos sistemas planetarios.

Objetivos de la Práctica

- Mapear una jerarquía de clases POO en una única tabla física (`InheritanceType.SINGLE_TABLE`).
- Implementar asociaciones `@ManyToOne` con carga perezosa controlada (`FetchType.LAZY`).
- Resolver retos tácticos mediante consultas avanzadas en HQL evitando consultas SQL nativas.
- Demostrar la mitigación empírica del problema de rendimiento N+1.

2. Fase 1: Infraestructura de Datos Desnormalizada (PostgreSQL)

Para implementar la estrategia de **Tabla Única**, toda la jerarquía de naves de la aplicación Java debe persistirse físicamente en una única tabla relacional denominada `naves`, complementada por la tabla `planetas` que modela las bases de despliegue.

2.1. Diseño del Esquema Relacional (DDL)

Esta tabla física cuenta con las siguientes particularidades obligatorias de la estrategia de tabla única:

- **Columna Discriminadora (`tipo_nave`):** Columna de tipo `VARCHAR(30)` no nula que almacena el identificador del subtipo (`'CAZA'` o `'DESTRUCTOR'`) para que el motor Hibernate instancie la subclase correcta en tiempo de ejecución.
- **Atributos Específicos Nulables:** Las columnas `potencia_fuego` (propia de los cazas) y `capacidad_escudos` (propia de los destructores) deben permitir valores `NULL`, dado que ninguna fila utilizará ambas columnas simultáneamente.
- **Integridad Referencial:** Clave foránea `fk_planeta` con comportamiento `ON DELETE SET NULL` para garantizar que la destrucción de una base no elimine el registro histórico de la nave.

PostgreSQL Console - DDL Desnormalizado de Base de Datos

```
antonio@starfleet-station:~/Examen$ psql -U antonio -d hangar_imperio_db -f schema_init.sql

-- Limpieza preventiva de entidades existentes
DROP TABLE IF EXISTS naves;
DROP TABLE IF EXISTS planetas;

-- Creación de la tabla independiente 'planetas'
CREATE TABLE planetas (
    planeta_id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL UNIQUE,
    clima VARCHAR(50),
    afiliacion VARCHAR(50) DEFAULT 'Neutral'
);

-- Creación de la tabla polimórfica 'naves' (Estrategia SINGLE_TABLE)
CREATE TABLE naves (
    nave_id SERIAL PRIMARY KEY,
    tipo_nave VARCHAR(30) NOT NULL,          -- Columna Discriminadora ('CAZA' / 'DESTRUCTOR')
    nombre VARCHAR(100) NOT NULL,
    modelo VARCHAR(50) NOT NULL,
    estado_sistema VARCHAR(20) NOT NULL,     -- 'Critico', 'Estable', 'Mantenimiento'
    planeta_origen_id INT,
    potencia_fuego INT,                      -- Específico de CazaLigero (adpite NULL)
    capacidad_escudos INT,                   -- Específico de DestructorEstelar (admite NULL)
    CONSTRAINT fk_planeta FOREIGN KEY (planeta_origen_id)
        REFERENCES planetas(planeta_id) ON DELETE SET NULL
);
✔ CREATE TABLE planetas OK
```

Figura 1: Creación de tablas en PostgreSQL con columna discriminadora y atributos desnormalizados.

2.2. Verificación de Datos Semilla

Se introdujeron los planetas (Coruscant, Yavin IV y Tatooine) con diferentes afiliaciones políticas, así como 5 naves de prueba combinando tipos y estados del sistema (`Estable`, `Critico` y `Mantenimiento`):

PostgreSQL Query Tool - Verificación de Semillas de Datos

```
antonio@starfleet-station:~/Examen$ psql -U antonio -d hangar_imperio_db
```

```
hangar_imperio_db=# SELECT * FROM planetas;
```

planeta_id	nombre	clima	afiliacion
1	Coruscant	Templado	Imperio
2	Yavin IV	Jungla	Alianza Rebelde
3	Tatooine	Árido	Neutral

(3 rows)

```
hangar_imperio_db=# SELECT nave_id, tipo_nave, nombre, modelo, estado_sistema, planeta_origen_id, potencia_fuego,
capacidad_escudos FROM naves;
```

nave_id	tipo_nave	nombre	modelo	estado_sistema	planeta_id	potencia_fuego	capacidad_escudos
1	CAZA	Caza X-Wing	T-65B	Estable	2	85	NULL
2	DESTRUCTOR	Destructor Estelar	Clase Imperial I	Estable	1	NULL	9500
3	DESTRUCTOR	Superdestructor Ejecutor	Clase Ejecutor	Critico	1	NULL	45000
4	CAZA	Caza TIE Alfa	TIE/ln	Critico	1	40	NULL
5	CAZA	Halcón Milenario	YT-1300	Mantenimiento	3	120	NULL

(5 rows)

Figura 2: Consulta tabular de verificación en PostgreSQL con las 5 naves y 3 planetas insertados.

3. Fase 2: Configuración del Motor ORM y Estructura

3.1. Gestión de Dependencias (Maven)

El proyecto utiliza Apache Maven para orquestar la compilación y gestionar las bibliotecas esenciales: Hibernate ORM Core 6.4.4 (especificación Jakarta Persistence) y el controlador JDBC oficial de PostgreSQL 42.7.2.

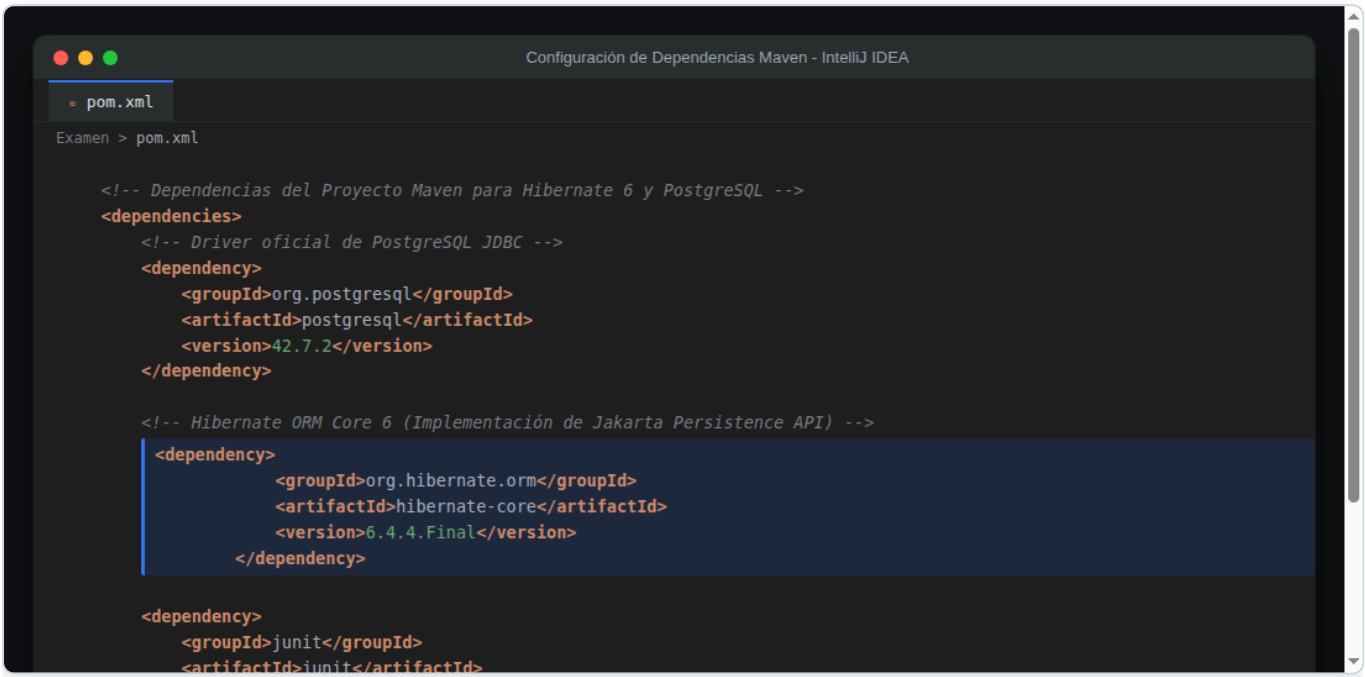


Figura 3: Dependencias del proyecto en el archivo pom.xml (Hibernate 6.4.4.Final y PostgreSQL Driver).

3.2. Descriptor Central de Hibernate (`hibernate.cfg.xml`)

El archivo de configuración establece las credenciales a la base de datos `hangar_imperio_db`, el dialecto optimizado para PostgreSQL y los parámetros de auditoría `show_sql` y `format_sql` para inspeccionar el SQL generado por detrás.

Descriptor de Configuración Hibernate - IntelliJ IDEA

hibernate.cfg.xml

src/main/resources > hibernate.cfg.xml

```xml
<hibernate-configuration>
    <session-factory>
        <!-- 1. Propiedades de conexión a PostgreSQL -->
        <property name="connection.driver_class">org.postgresql.Driver</property>
        <property name="connection.url">jdbc:postgresql://localhost:5432/hangar_imperio_db</property>
        <property name="connection.username">antonio</property>
        <property name="connection.password">admin</property>

        <!-- 2. Dialecto SQL para Hibernate 6 -->
        <property name="dialect">org.hibernate.dialect.PostgreSQLDialect</property>

        <!-- 3. Depuración e inspección de queries SQL generadas -->
        <property name="show_sql">true</property>
        <property name="format_sql">true</property>
        <property name="hbm2ddl.auto">update</property>

        <!-- 4. Registro de Entidades Persistentes -->
        <mapping class="org.dam.accesodatos.ut4.examen.models.Nave"/>
                <mapping class="org.dam.accesodatos.ut4.examen.models.CazaLigero"/>
                <mapping class="org.dam.accesodatos.ut4.examen.models.DestructorEstelar"/>
                <mapping class="org.dam.accesodatos.ut4.examen.models.Planeta"/>
    </session-factory>
</hibernate-configuration>
```

Figura 4: Configuración de SessionFactory y mapeo explícito de entidades en hibernate.cfg.xml.

3.3. Ciclo de Vida y Sesiones: `HibernateUtil.java`

La construcción de una `SessionFactory` es un proceso pesado en recursos que compila los metadatos y valida los esquemas. Por ello, se implementó el patrón **Singleton** a través de `HibernateUtil`, asegurando una única instancia durante la vida útil del proceso y permitiendo la invocación controlada de `shutdown()`.

4. Fase 3: Mapeo de Entidades y Jerarquía Polimórfica (POO / JPA)

4.1. Entidad Básica: `Planeta.java`

Mapea la tabla `planetas` mediante las anotaciones estándar de JPA: `@Entity`, `@Table(name="planetas")`, clave primaria con `@Id` y `@GeneratedValue(strategy = GenerationType.IDENTITY)`.

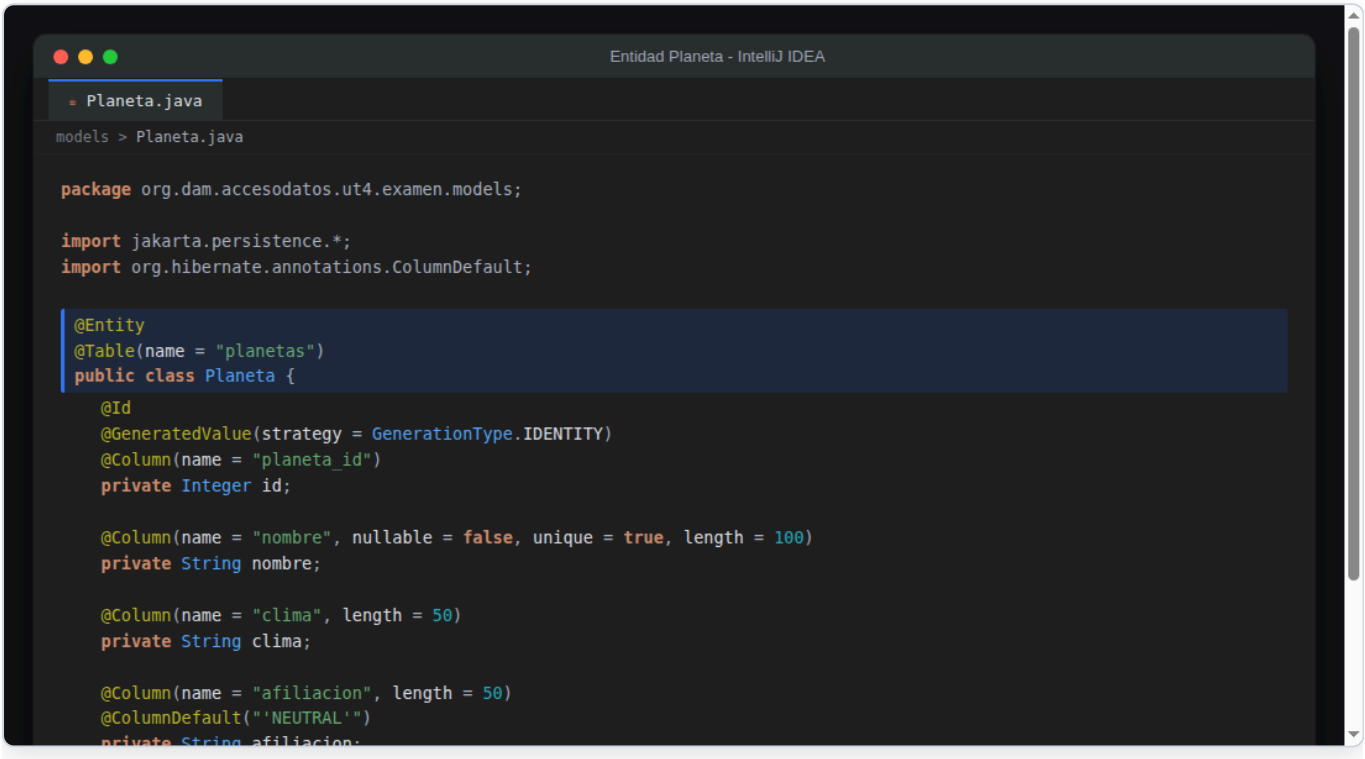


Figura 5: Mapeo de la entidad Planeta.java con restricciones de unicidad y no nulabilidad.

4.2. Raíz Jerárquica Abstracta: `Nave.java`

La clase abstracta `Nave` centraliza los atributos comunes (`nombre`, `modelo`, `estadoSistema`) y define la estrategia de herencia para el proveedor de persistencia:

- `@Inheritance(strategy = InheritanceType.SINGLE_TABLE)`: Define que todas las subclases compartirán la tabla `naves`.
- `@DiscriminatorColumn(name = "tipo_nave", discriminatorType = DiscriminatorType.STRING)`: Especifica el campo de distinción.
- `@ManyToOne(fetch = FetchType.LAZY)`: Configura la relación perezosa hacia `Planeta` a través de la columna `planeta_origen_id`. La carga perezosa evita traer datos del planeta hasta que sean explícitamente requeridos.

Mapeo Jerárquico Tabla Única - IntelliJ IDEA

Nave.java

models > Nave.java

```java
package org.dam.accesodatos.ut4.examen.models;

import jakarta.persistence.*;
import java.io.Serializable;

@Entity
@Table(name = "naves")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE) // Estrategia Tabla Única
@DiscriminatorColumn(name = "tipo_nave", discriminatorType = DiscriminatorType.STRING)
public abstract class Nave implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "nave_id")
    private int id;

    @Column(name = "nombre", nullable = false, length = 100)
    private String nombre;

    @Column(name = "modelo", length = 50, nullable = false)
    private String modelo;

    @Column(name = "estado_sistema", nullable = false, length = 20)
    private String estadoSistema;
```

Figura 6: Mapeo polimórfico de la clase base abstracta Nave.java con estrategia SINGLE_TABLE y carga LAZY.

4.3. Subclases Especializadas: `CazaLigero` y `DestructorEstelar`

Cada subclase extiende a `Nave` e implementa su respectivo `@DiscriminatorValue` junto con sus propiedades específicas:

Subclases Especializadas con Discriminador - IntelliJ IDEA

CazaLigero.java / DestructorEstelar.java

models > CazaLigero.java / DestructorEstelar.java

```java
// Subclase 1: CazaLigero (Discriminador: CAZA)
@Entity
@DiscriminatorValue("CAZA")
public class CazaLigero extends Nave {
    @Column(name = "potencia_fuego")
    private Integer potenciaFuego;
    // Constructores, Getters y Setters...
}

// Subclase 2: DestructorEstelar (Discriminador: DESTRUCTOR)
@Entity
@DiscriminatorValue("DESTRUCTOR")
public class DestructorEstelar extends Nave {
    @Column(name = "capacidad_escudos")
    private int capacidadEscudos;
    // Constructores, Getters y Setters...
}
```

@DiscriminatorValue CAZA & DESTRUCTOR UTF-8 | 4 spaces | LF

Figura 7: Mapeo de las subclases CazaLigero (CAZA) y DestructorEstelar (DESTRUCTOR).

5. Fase 4: Desafíos de Inteligencia Táctica (HQL Avanzado)

Todas las consultas se desarrollaron en la clase de servicio `HoloNetService`, empleando sesiones seguras con *try-with-resources* y parámetros con nombre (evitando inyecciones HQL).

5.1. Desafío 1: Sensor de Escuadrones Polimórfico (INNER JOIN + TYPE)

Firma: `public List<CazaLigero> obtenerCazasOperativosPorBando(String bando)`

Requisitos y Solución: Recupera únicamente cazas operativos de un bando. La consulta une la nave con su propiedad `planetaOrigen` mediante `INNER JOIN` y filtra por `TYPE(n) = CazaLigero`, descartando los cazas en estado `'Mantenimiento'`.

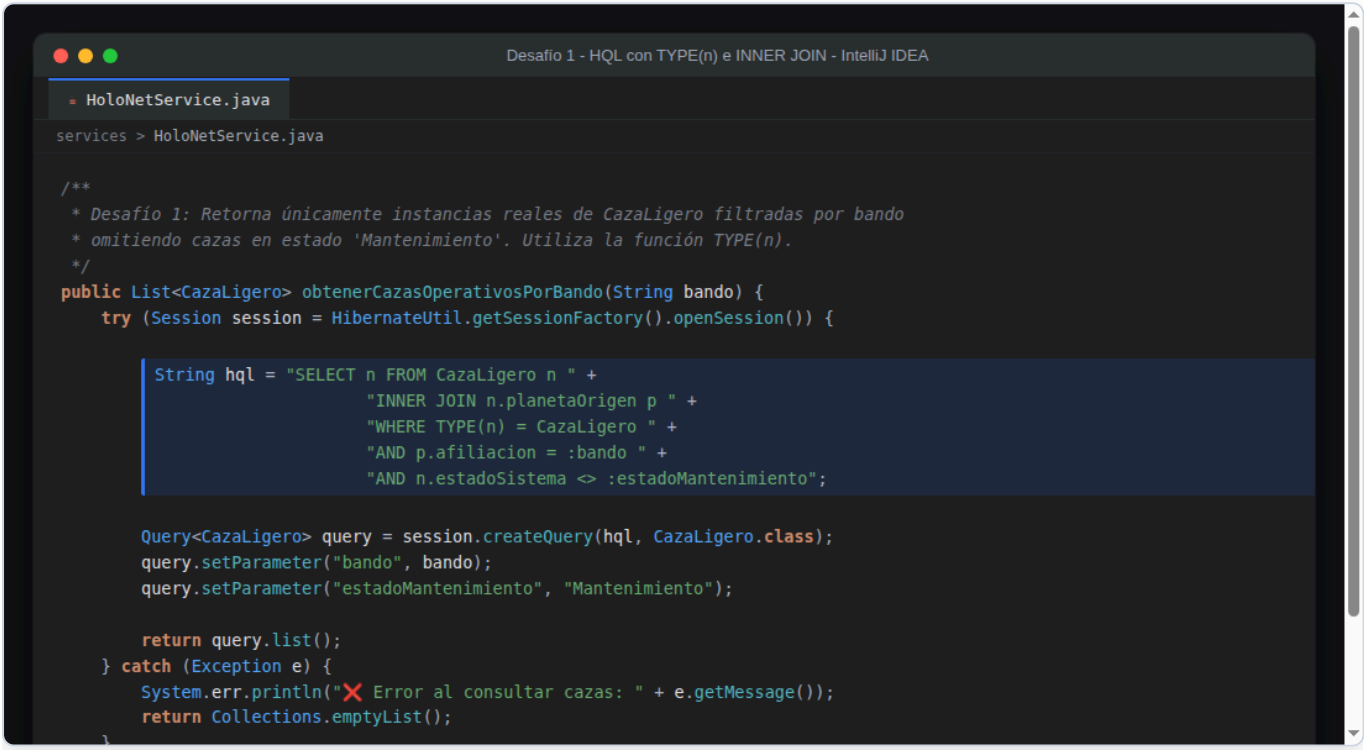


Figura 8: Implementación del Desafío 1 con discriminador polimórfico TYPE(n) e INNER JOIN en HQL.

5.2. Desafío 2: Mitigación de Consultas Recurrentes (FETCH JOIN)

Firma: `public List<Nave> obtenerFlotaCriticaPolimorfica()`

Requisitos y Solución: Recupera naves de cualquier tipo en estado `'Critico'`. Al estar configurada la relación como `FetchType.LAZY`, un acceso convencional causaría el **problema N+1** (1 consulta para la lista y N consultas secundarias para el planeta de cada nave). Se mitiga aplicando `INNER JOIN FETCH n.planetaOrigen p`, forzando la hidratación inmediata en una sola sentencia SQL.

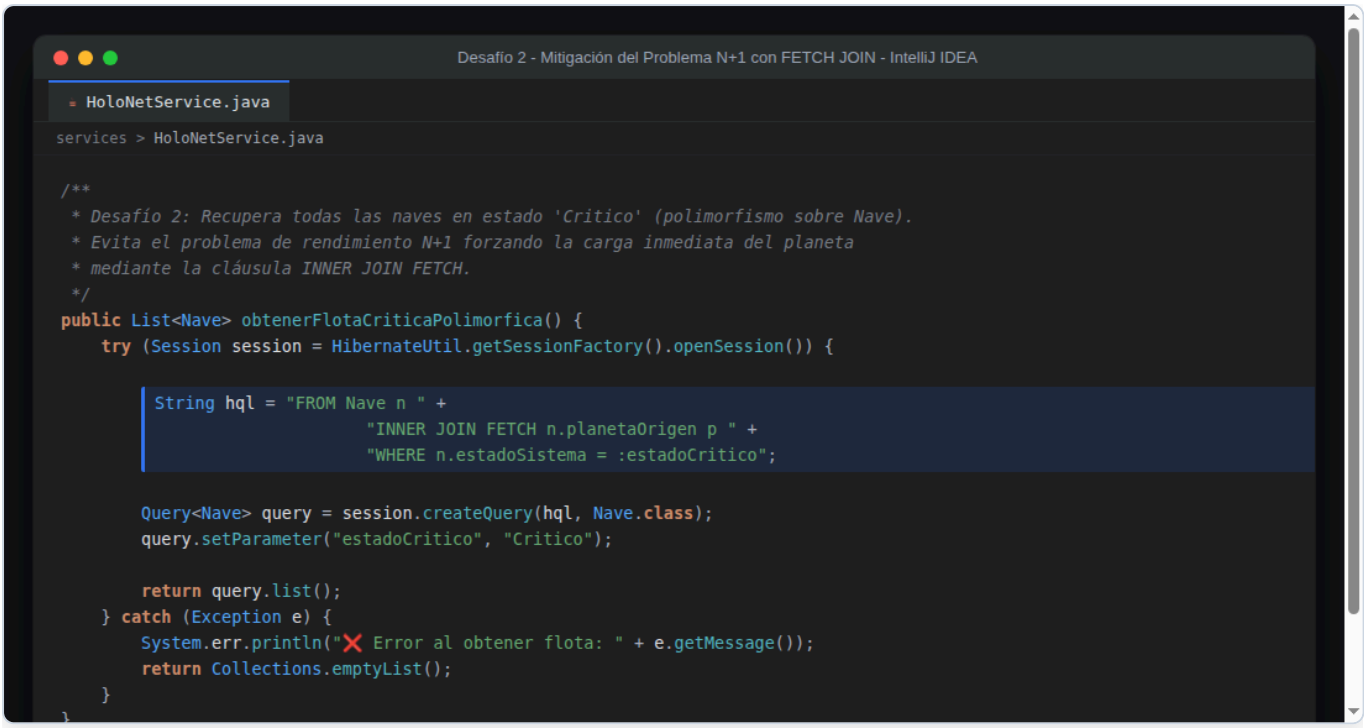


Figura 9: Desafío 2 con INNER JOIN FETCH eliminando el cuello de botella N+1.

5.3. Desafío 3: Proyecciones Planas de Inteligencia (LEFT JOIN)

Firma: `public List<Object[]> obtenerReporteGeneralFiltrado()`

Requisitos y Solución: Extrae un reporte escalar plano proyectando solo tres campos: nombre de la nave, clase concreta en el modelo POO (`TYPE(n)`) y nombre del planeta. El uso de `LEFT JOIN` asegura que las naves sin planeta asignado se incluyan con valor `null` .

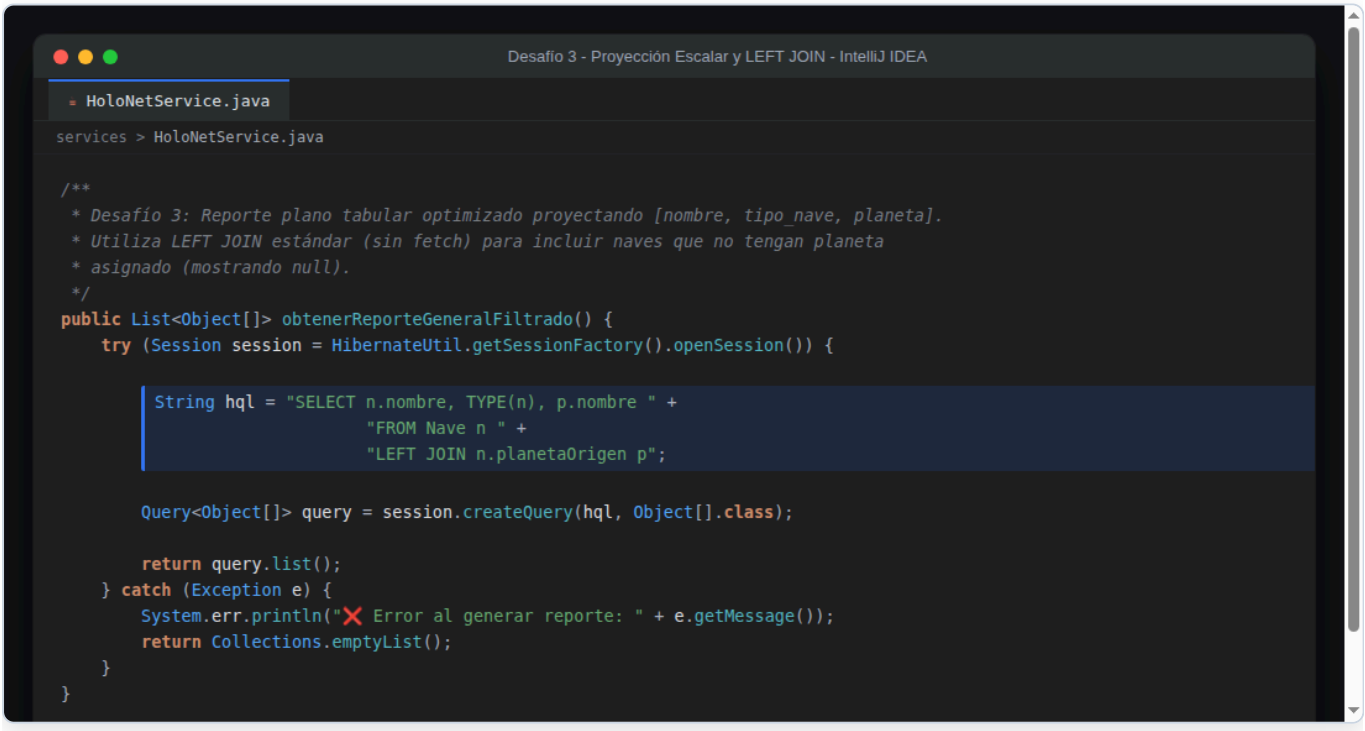


Figura 10: Desafío 3 con proyección escalar tabular (List<Object[]>) y LEFT JOIN.

6. Fase 5: Validación Experimental y Resultados en Consola

La clase de prueba `MainHangar` inicializa el contexto de Hibernate y ejecuta los tres desafíos de forma secuencial, permitiendo contrastar la salida en consola con las sentencias SQL reales generadas:

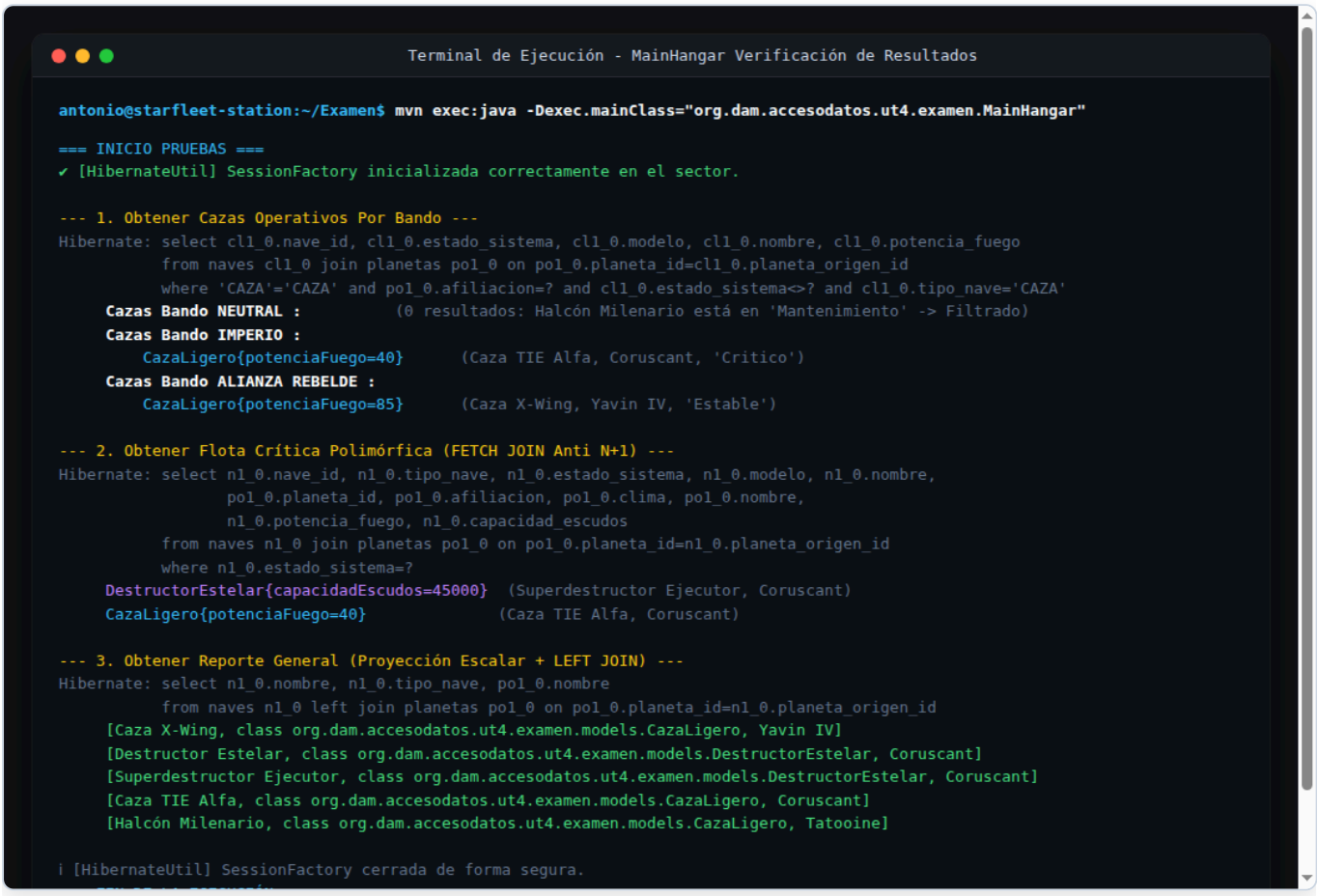


Figura 11: Ejecución completa en consola verificando la corrección funcional de los tres desafíos.

6.1. Análisis Detallado de Resultados

Desafío	Entrada / Bando	Resultado Obtenido	Verificación Técnica
1. Cazas Operativos	Neutral	0 naves	Halcón Milenario en Tatooine descartado por estado 'Mantenimiento'.
	Imperio	1 nave (Caza TIE Alfa)	En Coruscant, estado 'Critico' ≠ 'Mantenimiento'. Potencia: 40.

Desafío	Entrada / Bando	Resultado Obtenido	Verificación Técnica
	Alianza Rebelde	1 nave (Caza X-Wing)	En Yavin IV, estado 'Estable'. Potencia: 85.
2. Flota Crítica	Estado 'Critico'	Superdestructor Ejecutor (45000 escudos) y Caza TIE Alfa (40 potencia)	Carga polimórfica instantánea del planeta con 1 sola consulta SQL (Fetch Join).
3. Reporte General	Toda la flota	5 registros proyectados con clase Java y planeta origen	Proyección escalar optimizada de bajo impacto en memoria mediante Left Join.

7. Conclusiones Finales

1. **Rendimiento de SINGLE_TABLE:** Es la estrategia más veloz para lectura y operaciones polimórficas masivas, pues delega la distinción al motor mediante una simple columna discriminadora sin realizar `UNION` ni múltiples `JOIN` entre tablas secundarias.
2. **Poder Expresivo de HQL:** Permite consultar el modelo de objetos directamente respetando la herencia y polimorfismo mediante constructos como `TYPE(n)`.
3. **Control del Problema N+1:** La combinación de `FetchType.LAZY` por defecto con `FETCH JOIN` bajo demanda ofrece el equilibrio óptimo entre ligereza en reposo y eficiencia en consultas críticas.